

INOVAGAB

Plataforma de inovação corporativa — Grupo Águia Branca

Sprint 2 — Challenge FIAP 2026

Backend .NET 8 + MongoDB · migração do Firebase · insights de IA (Google Gemini) · app Android integrado

Integrantes (nome completo — RM)

- Athena Briana da Costa Freitas — RM 573457
- Gabriel Vasconcelos Auzier — RM 565926

Agenda

01

O que foi entregue

Do backend em Firebase para uma plataforma própria, ponta a ponta

02

Arquitetura do backend

Clean Architecture, .NET 8, MongoDB

03

Especificação da API

31 endpoints, contrato versionado

04

Segurança

JWT, perfis, rate limiting, hardening

05

Migração Firebase → MongoDB

Ferramenta própria, idempotente

06

Insights de IA (Gemini)

Modelo, guardrails, demonstração

07

App ↔ backend integrado

Fluxo real, de ponta a ponta

08

Qualidade e o que falta

Testes, cobertura, próximos passos

VISÃO GERAL

De um app com Firebase a uma plataforma própria

A Sprint 1 rodava direto contra o Firebase (Auth + Firestore), com regras de negócio no próprio app. A Sprint 2 substitui isso por um backend real — com autenticação, regras de negócio, relatórios e IA no servidor — e migra os dados existentes, sem quebrar o app.

31

endpoints REST

1.158

testes automatizados (backend)

101

testes automatizados (app)

~95%

cobertura Domain/Application

Backend

C# / .NET 8 · ASP.NET Identity + JWT · EF Core · MongoDB (transações, réplica)

IA

Google Gemini (generateContent) — insights sobre os resultados do dashboard

Migração

Firestore → MongoDB, ferramenta própria idempotente com relatório de conciliação

App

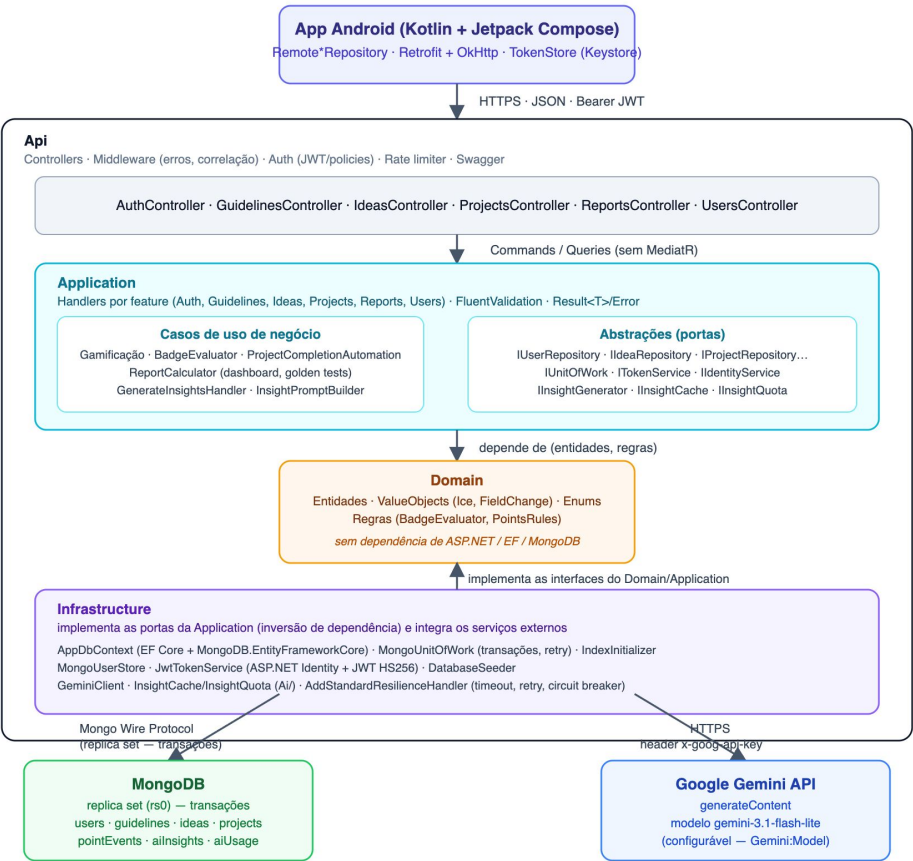
Kotlin + Jetpack Compose, agora 100% integrado à API própria (Firebase Auth/Firestore removidos)

ARQUITETURA

Api → Application → Domain ← Infrastructure

INOVAGAB — Arquitetura do backend (Sprint 2)

App Android → API .NET 8 (Clean Architecture) → MongoDB · Google Gemini



Regra de dependência: Api → Application → Domain ← Infrastructure. O Domain não conhece ASP.NET, EF Core, MongoDB nem HTTP (verificado por teste de arquitetura).

App fala só com a API

Nunca acessa Mongo ou Gemini diretamente — HTTPS + Bearer JWT

Regra de dependência

Api → Application → Domain ← Infrastructure (inversão de dependência)

Domain é puro

Sem ASP.NET, EF Core, MongoDB ou HTTP — verificado por teste de arquitetura

Sem MediatR

Um handler por caso de uso, injetado direto — menos indireção

MongoDB com transações

Réplica set (rs0); índices e TTL criados por um initializer no startup

ARQUITETURA

Por que essa organização

Domain

Entidades, regras (BadgeEvaluator, PointsRules), enums. Zero dependência de ASP.NET, EF Core, MongoDB ou HTTP — verificado por teste de arquitetura.

Application

Um handler por caso de uso, sem MediatR — rastro direto do controller até a regra de negócio. FluentValidation + Result<T>/Error.

Infrastructure

Implementa as portas da Application: EF Core + MongoDB, transações com retry, JWT/Identity, cliente do Gemini resiliente.

Sem MediatR: um time pequeno ganha mais navegando direto do controller até a regra do que com a indireção extra de pipeline behaviors.

CONTRATO DE API

31 endpoints, documentados a partir do OpenAPI real

Lista de endpoints gerados a partir da openapi.json (exportado do Swagger da API rodando) — rota, perfis, payload e resposta de exemplo, erros específicos. Os perfis vêm da matriz de autorização testada no backend.

Recurso	Rotas	Perfis	Exemplo
Auth	login · refresh · logout · me	público / autenticado	POST /auth/login → accessToken + refreshToken
Orientações	list · get · create · update · delete · history	autenticado / LIDER escreve	POST /guidelines (título, pilar, campanha)
Ideias	list · get · create · update · delete · ice · approve · reject	OPERADOR/GESTOR cria · GESTOR avalia	POST /ideas/{id}/approve → cria projeto, +50 pts
Projetos	list · get · updates · create · update · delete	GESTOR escreve · GESTOR/LIDER lê	PUT /projects/{id} → diff + version (409 se divergir)
Relatórios	summary · guidelines · guidelines/{id} · projects/{id}	LIDER	GET /reports/summary?period=&division=
IA	insights	LIDER	POST /reports/insights {period, refresh}
Usuários	list · ranking	GESTOR/LIDER / autenticado	GET /users/ranking → top do mês

SEGURANÇA

Autenticação, perfis e endurecimento

Autenticação

JWT HS256 (30 min) + refresh token opaco rotativo (7 dias). Reuso de um refresh já trocado revoga a família inteira.

Hardening (B21)

CORS restrito a origens configuradas · limite de payload (413) · headers de segurança em toda resposta · HSTS em produção.

Perfis (policies)

GestorOnly · LiderOnly · CanCreateIdea · ProjectsRead · UsersRead — testados numa matriz de 31 rotas × 4 identidades (124 casos).

Segredos

Chave JWT e chave do Gemini só por variável de ambiente; nunca em log; scan-secrets.sh varre o repositório antes de cada commit.

Senhas e força bruta

PBKDF2 (ASP.NET Identity); 5 tentativas erradas bloqueiam a conta por 15 min; rate limit de 10 req/min por IP em /auth/*.

Erros padronizados

application/problem+json com code estável e traceld; nenhuma resposta expõe stack trace.

FUNCIONALIDADE DE IA (PLUS)

Insights sobre os resultados do dashboard

Funcionalidade escolhida: gerar, sob demanda, uma leitura executiva dos mesmos números do dashboard — resumo, destaques, riscos e recomendações priorizadas — para a liderança decidir mais rápido.

Modelo: gemini-3.1-flash-lite (configurável)

POST /reports/insights · LIDER

Sem PII

Só agregados e títulos truncados (80 car.) — nunca nome, e-mail ou id de usuário. Testado capturando o payload real.

Anti-injeção

Textos livres viram dado dentro de um bloco delimitado; orientações vão por referência (G1...), nunca por id.

Resiliente

Timeout 20s por tentativa, 1 retry, circuit breaker. Falha → 503, nunca um insight inventado.

Cache + cota

6h por hash dos dados; refresh=true força novo. 6/min por usuário + teto diário atômico (aiUsage).

Limitações assumidas: free tier pequeno e instável (créditos podem se esgotar) · só português, tom executivo · a IA pode errar dentro do próprio schema — por isso o app sempre mostra “Gerado por IA — valide antes de decidir”.

FUNCIONALIDADE DE IA (PLUS)

Demonstração — resposta real do Gemini

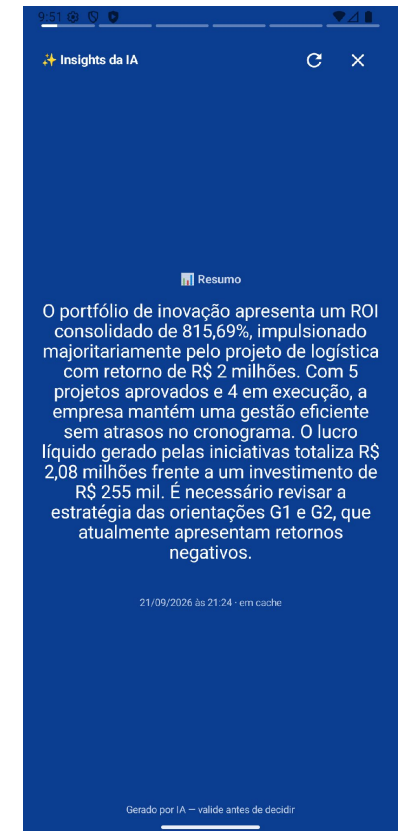
Chamada real, ambiente de desenvolvimento (dados do seed), ~2,7 s, modelo gemini-3.1-flash-lite:

```
"summary": "O portfólio de inovação apresenta um ROI consolidado de 36,73%, com um lucro líquido de 90 mil reais gerado a partir de um investimento total de 245 mil reais. A eficiência operacional na logística destaca-se como o principal motor de retorno financeiro."

"risks": ["ROI negativo de 68,75% no projeto de sustentabilidade (G2)", "Alta concentração de valor em um único projeto de alta performance"]

"recommendations[0]": { "title": "Revisão do projeto de sustentabilidade", "priority": "ALTA", "relatedGuidelineId": "665f..." }

"model": "gemini-3.1-flash-lite", "fromCache": false
```



APP ↔ BACKEND

Fluxo integrado, de ponta a ponta

App Android (Kotlin + Compose) → API .NET 8 real, no emulador, contra o backend em Docker. Sem mocks.



1. Dashboard — dados de /reports/summary, card de IA no topo



2. Apresentação — botão gerar insights



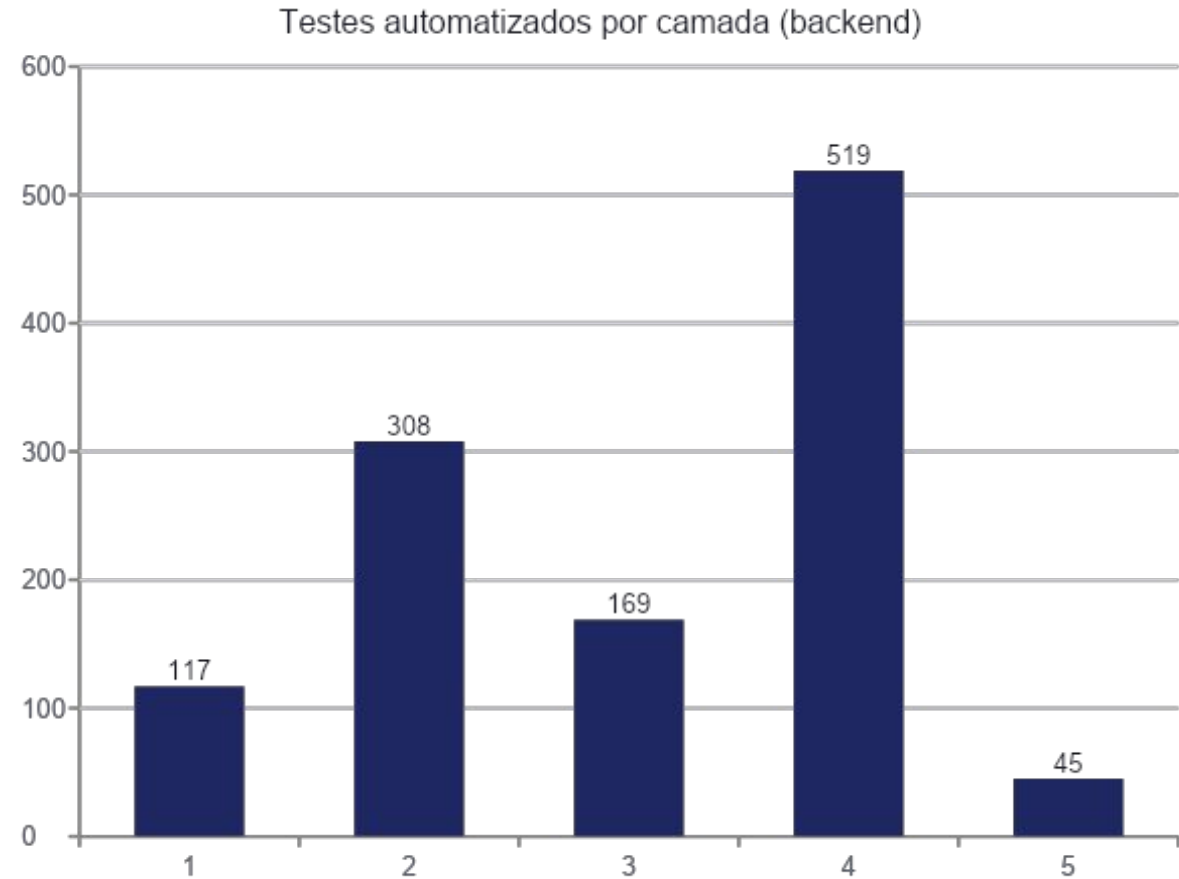
3. Reabrir mostra o último salvo (cache do servidor)

Sessão:
tokens JWT cifrados (Keystore) · refresh automático em 401

Dados:
"Tempo real" por polling de 15s + invalidação imediata após cada escrita

QUALIDADE

Testado em cada camada



- ✓ **1.158 testes no backend**
unitários + integração (Testcontainers e Mongo local) — os dois modos ficam verdes
- ✓ **101 testes no app**
MockWebServer/Turbine — refresh de token, mapeamento de erro, repositórios, ViewModels
- ✓ **Cobertura Domain/Application**
94,6% / 94,9% de linhas (meta era $\geq 80\%$)
- ✓ **Golden tests do dashboard**
3 conjuntos calculados à mão a partir das regras do Kotlin original
- ✓ **Concorrência testada de verdade**
aprovações e conclusões simultâneas, teto diário de IA sob 10 requisições paralelas

STATUS

O que está pronto e o que falta

Pronto

- Backend: Módulos (auth, orientações, ideias, projetos, relatórios, IA, hardening, matriz de autorização, migrador)
- App: Migração das chamadas Firebase para a API e adição de nova funcionalidade de insights com IA (insights no topo, apresentação em stories) — Firebase Auth/Firestore removidos
- Docker compose local validado de ponta a ponta (Mongo + API + seed)

Pendente

- Deploy público da API (B24/OP-5) — fallback documentado: docker compose + IP da LAN
- APK de release (M11) — depende da URL HTTPS do deploy

Obrigado.

INOVAGAB — Grupo Águia Branca · Sprint 2 · Challenge FIAP 2026

Repositório: <https://github.com/athenabriana/aguia-branca>